

Reviewing fundamental concepts in Python

Functions and conditional statements

CS 22B, Spring 2026

Module 01: Essential

Presented by Jessica Huynh-Westfall, Ph.D.

AGENDA



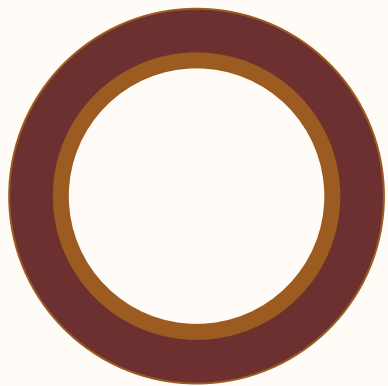
Writing functions

- Testing function



Conditional statement

- if, elif, else
- and, or, not



Chapter 5: Writing our own function

Creating your own function allows you to re-use the same code multiple times within a program.

Define a function

- inputs - function arguments
- output - return value



Write a function to calculate AT content

04

```
def get_at_content(sequence):  
    at_content =  
    (sequence.count("A")+sequence.count("T"))/len(sequence)  
    return(at_content)
```

where sequence is **input argument** and at_content is **return output value**

```
def get_at_content(sequence):  
    ..  
    ..
```

define function

argument

function body

```
at_content = get_at_content("ATGTTTCGATCGTATCG")
```

store return value

calling function

Some notes about **functions**

05

- Function can be define with no arguments
 - Not super useful

```
def what_number():  
    return 24
```

Some notes about **functions**

06

- Function can be define with no arguments
 - Not super useful
- Function do not have to return a value

```
my_num = 42
def what_number(my_num):
    double_me = my_num*2
    print(string(double_me))
```

If the return statement has no arguments, the function automatically returns the value `None`

Some notes about **functions**

07

- Function can be define with no arguments
 - Not super useful
- Function do not have to return a value
- Function can use keyword arguments

```
def get_at_content(sequence, sig_figs):  
    at_content = (sequence.count("A")+sequence.count("T"))/len(sequence)  
    return round(at_content, sig_figs)
```

```
get_at_content("ATATATGCTGCATCGA", 2)
```

```
get_at_content(sig_figs=2, sequence="ATATATGCTGCATCGA")
```

Some notes about **functions**

08

- Function can be define with no arguments
 - Not super useful
- Function do not have to return a value
- Function can use keyword arguments
- Function argument can have default value

```
def get_at_content(sequence, sig_figs=2):  
    at_content = (sequence.count("A")+sequence.count("T"))/len(sequence)  
    return round(at_content, sig_figs)
```

```
get_at_content("ATATATGCTGCATCGA")
```


Class exercise

09

Given sequence

```
dna = "ATTGGCTATACCGGCATCGGGTAGCC"
```

CL 4.1:

Write a function that takes two arguments; a DNA sequence and a nucleotide (ie., "A", "T", "C" or "G"). Return the percentage of the DNA sequence that the nucleotide makes up.

* Challenge, pass in 3rd argument that returns value with 2 significant figure

Using assert to test function

You want to check that your code does what you intended it to do.

assert - built-in tool run code on test input and compare it to expected answer

```
assert get_at_content("ATCG") == 0.5
```



What if an assertion is false?

11

```
>>> assert get_at("ATCG") == 0.3
Traceback (most recent call last):
  File "<python-input-31>", line 1, in <module>
    assert get_at("ATCG") == 0.3
           ^^^^^^^^^^^^^^^^^^^^^
AssertionError
```

If assertion turns out to be false, program will stop and get **AssertionError**.

Class exercise

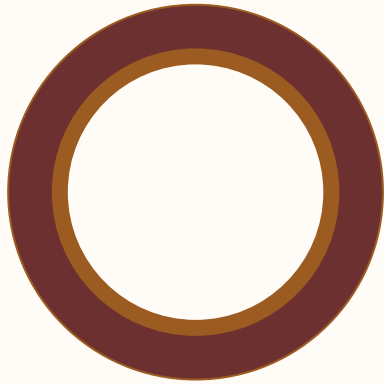
12

Given sequence “AAATTTCCCGGG”

CL 4.2

Write assertion statements to test your function, `get_percent_2sig`.

- Write assertion that takes in sequence and nucleotide [“A”]
- If you change the assertion so that the expected result is 0.80, what is your output?



Chapter 6: Conditional tests

Writing programs that act as decision-makers using conditional statements.

condition - code that evaluates different conditions and return True or False

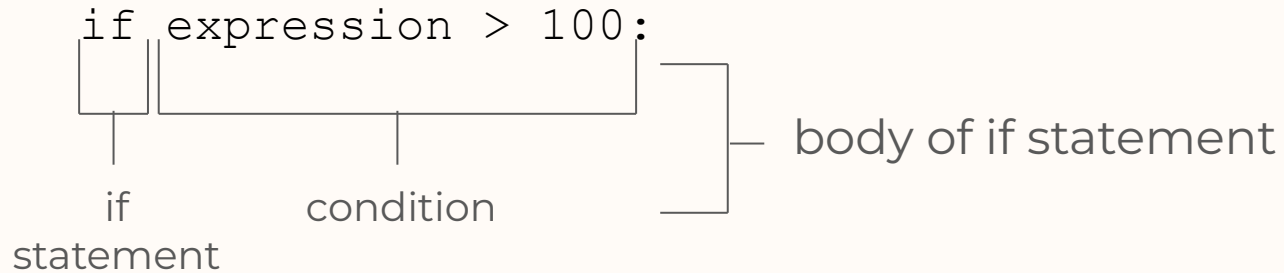


Examples of condition statements

14

```
print(3 == 5)
print(3 > 5)
print(len("ATGC") > 5)
print("GAATTC".count("T") > 1)
print("ATGCTT".startswith("ATG"))
print("ATGCTT".endswith("TTT"))
print("ATGCTT".isupper())
print("ATGCTT".islower())
print("V" in ["V", "W", "L"])
```

```
expression = 150  
  
if expression > 100:  
    print("The expression is high")
```



if statements

Class exercise

16

```
accession_name = ['ab56', 'bh84', 'hg84', 'ay97', 'cd72',  
'ef56']
```

CL 4.3:

Find the accession that starts with 'a'

1. Write a for loop that iterates over the accession_name list
2. Use a conditional statement to ask if the accession name starts with 'a' and if it does, print out the name

else statements

17

Allows us to have more than two possible branches.

Example: We want to write three files of accession names; one start with 'a', one start with 'b', and all others.

pseudocode

Create 3 files

for each name in list:

- if name starts with character 'a'

 - add name to list a

- elif name starts with character 'b'

 - add name to list b

- else add name to last list



Class exercise

18

```
accession_name = [ 'ab56', 'bh84', 'hg84', 'ay97', 'cd72',  
                   'ef56' ]
```

CL 4.4:

Write three files of accession names; one start with 'a', one start with 'b', and last one has all others accession names.

while loops

19

Use while with condition to determine when to exit a loop

- Iterate over collection of items and run until some condition is met



```
count = 0  
while count < 10:  
    print(count)  
    count += 1
```

boolean operators

and, or, not

```
accession_name = ['ab56', 'bh84', 'hg84', 'ay97', 'cd72',  
                  'ef56']
```

```
for name in accession_name:  
    if name.startswith('a'):  
        if name.endswith('7'):  
            print(name)
```

Use of **and** is equivalent to two nested **if** statements

```
for name in accession_name:  
    if name.startswith('a') and name.endswith('7'):  
        print(name)
```

boolean operators

and, **or**, not

Use of **or** is equivalent to joining two+ conditions

```
accession_name = ['ab56', 'bh84', 'hg84', 'ay97', 'cd72', 'ef56']

for name in accession_name:
    if name.startswith('a') or name.startswith('b'):
        print(name)
```

```
for name in accession_name:
    if (name.startswith('a') or name.startswith('b'))and name.endswith('7'):
        print(name)
```

boolean operators

and, or, **not**

You can negate any type of condition by prefixing it with the word **not**

```
accession_name = ['ab56', 'bh84', 'hg84', 'ay97', 'cd72',  
                  'ef56']  
  
for name in accession_name:  
    if name.startswith('a') and not name.endswith('7'):  
        print(name)
```

Class exercise

18

CL 4.5:

Given the file, `drosophila.csv`, write a function called `'get_gene()'` that takes in the data file. Write a loop that iterates through the rows and filter out the gene names that belong to *Drosophila melanogaster* or *Drosophila simulans*. Returns the values to a new list `'Dm_Ds'`